Linear vs. Nonlinear Predictors from Scratch

CS109 Project
Winter 2026

Maleeka Raddygala
maleeka@stanford.edu

Code Repository: `github.com/NullPointerNinja007/Linear_vs_non_linear_predictors_from_scratch`

Project Overview

This project compares a linear probabilistic classifier, **softmax logistic regression**, with a **shallow neural network** implemented entirely from scratch using NumPy. The goal was to study when a nonlinear model provides a meaningful improvement over a linear model, rather than assuming that a neural network is always better by default.

Two datasets were used:

- **UCI Heart Disease Dataset**
- **UCI Human Activity Recognition (HAR) Dataset**

The softmax regression model represents a linear decision boundary, while the shallow neural network adds one hidden layer with a ReLU activation, allowing nonlinear feature interactions.

Impact

This project compares linear and nonlinear predictors implemented entirely from scratch using NumPy. In particular, we study multiclass softmax logistic regression and a one-hidden-layer neural network on two datasets: the UCI Heart Disease dataset and the UCI Human Activity Recognition dataset. The goal is to understand when a simple linear model is sufficient and when a nonlinear model provides additional predictive power.

These tasks are relevant to real-world health applications. Heart disease prediction can support clinical risk assessment and early screening, while human activity recognition has applications in wearable health monitoring, rehabilitation tracking, fall detection, and assistive technologies. By comparing interpretable linear models with more expressive nonlinear models, this project also examines the tradeoff between model simplicity, performance, and practical usefulness.

Models

Softmax Regression

For multiclass classification, the model predicts

$$P(y = k \mid x) = \frac{e^{\theta_k^T x}}{\sum_{j=1}^{K} e^{\theta_j^T x}},$$

and parameters are trained by minimizing cross-entropy loss.

Shallow Neural Network

The nonlinear model uses one hidden layer:

$$h = \mathrm{ReLU}(W_1 x + b_1),$$

$$P(y = k \mid x) = \mathrm{softmax}(W_2 h + b_2).$$

Experimental Results

1. Heart Disease Dataset

For the heart disease dataset, both models were evaluated using cross-validation. Surprisingly, the linear model slightly outperformed the shallow neural network.

Model	**Mean Accuracy**	**Std. Dev.**
Shallow Neural Network	0.7274	0.0176
Softmax Regression	0.7341	0.0334

This suggests that for the heart disease dataset, the feature space is already structured enough that a linear classifier is highly competitive. Since I expected the neural network to perform better, this motivated moving to a larger and more complex dataset.

2. Human Activity Recognition Dataset (with PCA)

The HAR dataset is much larger and has 561 engineered features extracted from smartphone sensor signals. Because I was training on my own machine without a dedicated GPU, I first applied PCA to reduce dimensionality and shorten training time.

After PCA-based dimensionality reduction, softmax regression again slightly outperformed the shallow neural network.

Model	**Test Accuracy**
Shallow Neural Network	0.9084
Softmax Regression	0.9128

This indicates that after linear compression of the feature space, much of the class structure remained nearly linearly separable. In other words, PCA made the reduced representation more favorable to the linear model.

3. Human Activity Recognition Dataset (Original Feature Space)

To understand whether nonlinear structure was being removed by PCA, I trained both models on the full HAR dataset without dimensionality reduction.

Model	Test Accuracy
Softmax Regression	0.9464
Shallow Neural Network	0.9569

This was the most interesting result in the project. On the original 561-dimensional feature space, the shallow neural network finally outperformed softmax regression. This suggests that the full feature representation contains nonlinear structure that is not fully captured by a purely linear model.

Hyperparameter Search for the Neural Network

The neural network did not outperform softmax regression immediately. In fact, on early experiments it underperformed badly. To investigate this, I performed a focused hyperparameter search over hidden layer width, learning rate, and number of gradient descent iterations.

The final search region was:

$$H \in \{32, 48, 64\}, \qquad \eta = 0.01, \qquad T \in \{15000, 20000, 30000\}.$$

The results were:

Hidden Units	Learning Rate	Iterations	Test Accuracy
32	0.01	15000	0.9518
32	0.01	20000	0.9549
32	0.01	30000	**0.9569**
48	0.01	15000	0.9532
48	0.01	20000	0.9559
48	0.01	30000	0.9549
64	0.01	15000	**0.9569**
64	0.01	20000	0.9498
64	0.01	30000	0.9532

The best configuration was:

$$(H, \eta, T) = (32, 0.01, 30000),$$

which achieved

$$\text{Accuracy} = 0.9569.$$

A few trends were visible in this search:

- Increasing the number of iterations consistently helped the 32-unit network.
- Moderate hidden sizes performed best; increasing width beyond 32 did not guarantee further improvement.

- The 64-unit network reached the same peak accuracy at 15000 iterations, but was less stable as training continued.

This suggests that the HAR dataset contains meaningful nonlinear structure, but that a *moderately sized* network is sufficient to exploit it. A larger network did not automatically improve performance, likely because the extra capacity made optimization less stable or increased variance.

Discussion

The results show a clear progression:

- On the **heart disease dataset**, softmax regression slightly outperformed the shallow neural network.
- On the **HAR dataset after PCA**, the linear model again remained slightly stronger.
- On the **original HAR feature space**, after careful hyperparameter tuning, the shallow neural network achieved the best overall result.

This pattern suggests that model complexity only becomes beneficial when the dataset retains enough rich structure for nonlinear interactions to matter. When the feature space is compact or already close to linearly separable, the simpler linear model can be just as strong or even better.

The hyperparameter search was also important. A poorly tuned neural network underperformed the linear baseline, but a well-tuned one was able to surpass it. This reinforces a useful lesson: nonlinear models are not inherently superior; they require appropriate optimization and capacity to realize their advantage.

Conclusion

This project demonstrates that nonlinear models do not automatically outperform linear models. Instead, performance depends strongly on the structure of the data, the richness of the feature representation, and the quality of the optimization.

Across these experiments:

- **Softmax regression** was a very strong baseline on both datasets.
- **Shallow neural networks** required more tuning, but were able to outperform the linear model on the full HAR dataset.
- **Dimensionality reduction** changed the balance between the two models by making the classification task more favorable to linear methods.

Overall, the project shows that understanding the data is just as important as increasing model complexity.

Forward and Backward Passes

This section explains how both models compute predictions and how their parameters are updated.

1. Softmax Regression

For an input vector $x \in \mathbb{R}^d$ and K classes, softmax regression computes a score for each class:

$$z = \Theta^T x,$$

where $\Theta \in \mathbb{R}^{d \times K}$ is the parameter matrix.

The predicted probability for class k is then

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

Thus, the forward pass consists of:

1. computing the class scores z,
2. applying softmax to convert scores into probabilities.

For a dataset of n examples, the cross-entropy loss is

$$L(\Theta) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} Y_{ik} \log P_{ik},$$

where Y is the one-hot encoded label matrix.

The key simplification is that for softmax combined with cross-entropy,

$$\frac{\partial L}{\partial Z} = \frac{1}{n}(P - Y).$$

From this, the gradient with respect to the parameter matrix is

$$\frac{\partial L}{\partial \Theta} = X^T \frac{(P - Y)}{n}.$$

So the backward pass for softmax regression is simply:

1. compute prediction error $P - Y$,
2. propagate that error directly to the weights,
3. update Θ by gradient descent.

2. Shallow Neural Network

The shallow neural network adds one hidden layer before the final softmax classifier.

Let

$$X \in \mathbb{R}^{n \times d}, \quad W_1 \in \mathbb{R}^{d \times H}, \quad b_1 \in \mathbb{R}^H,$$
$$W_2 \in \mathbb{R}^{H \times K}, \quad b_2 \in \mathbb{R}^K,$$

where H is the number of hidden neurons.

Forward Pass

The network computes:

$$Z_1 = XW_1 + b_1,$$
$$A_1 = \mathrm{ReLU}(Z_1),$$
$$Z_2 = A_1 W_2 + b_2,$$
$$P = \mathrm{softmax}(Z_2).$$

So the forward pass is:

1. compute a linear transformation into the hidden layer,
2. apply the ReLU nonlinearity,
3. compute output layer scores,
4. apply softmax to obtain class probabilities.

The loss is again the cross-entropy loss:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} Y_{ik}\log P_{ik}.$$

Backward Pass

As with softmax regression, the output-layer error is

$$\delta_2 = \frac{1}{n}(P - Y).$$

The gradients for the output layer are:

$$\frac{\partial L}{\partial W_2} = A_1^T \delta_2,$$

$$\frac{\partial L}{\partial b_2} = \sum_{i=1}^{n} (\delta_2)_{i,:}.$$

To backpropagate into the hidden layer, we compute

$$\frac{\partial L}{\partial A_1} = \delta_2 W_2^T.$$

Since the hidden activation is ReLU,

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \le 0 \end{cases}$$

and therefore

$$\delta_1 = (\delta_2 W_2^T) \odot \mathbf{1}_{Z_1 > 0},$$

where $\odot$ denotes elementwise multiplication.

The gradients for the first layer are then

$$\frac{\partial L}{\partial W_1} = X^T \delta_1,$$

$$\frac{\partial L}{\partial b_1} = \sum_{i=1}^{n} (\delta_1)_{i,:}.$$

The backward pass can therefore be summarized as:

1. compute output-layer error $\delta_2 = \frac{1}{n}(P - Y)$,
2. compute gradients for W_2 and b_2,
3. propagate error backward through W_2,
4. apply the derivative of ReLU,
5. compute gradients for W_1 and b_1,
6. update all parameters using gradient descent.

Why the Neural Network Can Do More

Softmax regression learns only a linear mapping from input features to class probabilities. The neural network instead first transforms the data through a hidden layer and a nonlinearity:

$$x \mapsto \mathrm{ReLU}(W_1 x + b_1).$$

This allows the network to model nonlinear interactions between features. In principle, this gives it greater expressive power than softmax regression. However, as seen in the experiments, whether this additional flexibility helps depends strongly on the structure of the data and the feature representation.